

Chapter 6: Fixed-Size Arrays and Array-Based Containers

data structures · arrays · heap memory · the Big Three · valgrind

HOW TO READ THESE NOTES

- **NOT TESTED** boxes with a dashed gold border contain material you are **not responsible for** in this class. Included for completeness only.
- Not tested in this chapter: **using** `std::array<>` (6.2.2), **how** `memcpy()` **works** (6.5), and **all of move semantics** (6.9, including the move constructor and move assignment operator).
- Everything else is fair game.

6.1 Introduction to Data Structures

- Two important considerations when designing a computational approach to a problem: the **algorithm** and the **data structures** used to implement it.
- **Algorithm**: a well-defined set of instructions that can be followed to solve a problem.
- **Data structure**: a collection of data, the relationships among them, and the functions/operations that can be applied to those data values.
 - Each data structure provides an **interface** defining how its data can be accessed.
 - It organizes data in memory in a way that efficiently supports that access.
- Performance depends not only on an algorithm's implementation but also on the data structures it is used on.
- **Container**: a type of data structure used to store other objects. A container **manages the memory** used by the objects it holds.

C++ standard library container categories

Category	Definition	Examples (chapter introduced)
Sequence container	Allows its elements to be accessed sequentially.	<code>std::array<></code> (6), <code>std::vector<></code> (7), <code>std::list<></code> (8), <code>std::forward_list<></code> (8), <code>std::deque<></code> (9)
Associative container	Stores data in sorted order ; elements can be searched in $\Theta(\log(n))$ time.	<code>std::map<></code> (18), <code>std::multimap<></code> (18), <code>std::set<></code> (18), <code>std::multiset<></code> (18)
Unordered associative container	Elements have no defined order . Usually implemented with hash tables; fast lookup (average-case $\Theta(1)$).	<code>std::unordered_map<></code> (17), <code>std::unordered_multimap<></code> (17), <code>std::unordered_set<></code> (17), <code>std::unordered_multiset<></code> (17)
Container adaptor	An interface adapted for a specific purpose; usually restricts the functionality of a standard container.	<code>std::queue<></code> (9), <code>std::stack<></code> (9), <code>std::priority_queue<></code> (10)

- This chapter starts with the most basic sequential container: the **fixed-size array**.

6.2 Array Fundamentals

6.2.1 C-style arrays

- **Array**: a **fixed-size** container of objects stored **contiguously** in memory. Every element must have the **same type**.

```

1 int32_t arr[] = {0, 1, 5, 3}; // initializer list: array of size 4
2 arr[2] = 2;
3 int32_t* ptr = arr;           // ptr points to the FIRST element
4 ptr = &arr[1];

```

- **Line 1:** initializes the array with an initializer list; `arr` has size 4 with values 0, 1, 5, 3.
- **operator[]** accesses or modifies any element in **$O(1)$** time, because `arr[i]` does the same thing as `*(arr + i)` (pointer arithmetic plus a dereference), which is constant time.
- **Line 3 / array decay:** assigning `arr` to a pointer assigns a **pointer to the first element**. With a few exceptions (such as `sizeof()`), an array expression **decays into a pointer to its first element** on assignment or when used as a function argument.
- **No bounds checking.** Error checking is the programmer's responsibility, because automatic checking adds performance overhead that may not be wanted when speed matters. You must add any checks you need yourself.
- The array above is **non-dynamic**: its size is fixed **at compile time**. Inserting more elements requires reallocating the data into a bigger array elsewhere in memory.

REMARK: UNIFORM INITIALIZATION (C++11)

- C++11 introduced **uniform initialization**: one common curly-brace syntax that can initialize anything from primitives to large objects, e.g. `int32_t num{281}`.
- Two notable types of initialization: **default** and **value** initialization. Atomic types are **undefined** when default initialized; curly braces value-initialize them.

Default initialization	Value initialization
<code>int32_t x;</code> x is undefined	<code>int32_t x{};</code> x is 0
<code>int32_t* y;</code> y is undefined	<code>int32_t* y{};</code> y is nullptr
<code>int32_t z[5];</code> z has undefined data	<code>int32_t z[5]{};</code> z is initialized with 0's

- Curly braces **cannot** be used in narrowing initializations. `int32_t num = 281.57` narrows from double to int by truncating (`num = 281`), but `int32_t num{281.57}` is a **compiler error**. The same applies to arrays.

6.2.2 The STL array (std::array)

NOT TESTED — you do not need to know how to use std::array<> in this class

(You may need it in upper-level classes.)

```

std::array<TYPE, SIZE> arr;

std::array<int32_t, 4> arr = {0, 1, 5, 3}; // size 4 holding 0, 1, 5, 3
std::array<int32_t, 4> arr{0, 1, 5, 3};   // curly brace initialization

```

- `std::array<>` lives in the `<array>` library and is a **wrapper over the C-style array** with features the C array lacks.
- It gives access to member functions (such as `.size()`) and iterators (chapter 11).

Function	Behavior
<code>.size()</code>	Returns the size of the array
<code>.empty()</code>	Returns a Boolean specifying whether the array is empty
<code>.front()</code>	Returns the first element of the array
<code>.back()</code>	Returns the last element of the array
<code>.begin()</code>	Random access iterator to the first element
<code>.end()</code>	Random access iterator to the position one past the last element
<code>.cbegin()</code>	Constant random access iterator to the first element
<code>.cend()</code>	Constant random access iterator to one past the last element
<code>.rbegin()</code>	Reverse iterator to the last element
<code>.rend()</code>	Reverse iterator to the position one before the first element
<code>.crbegin()</code>	Constant reverse iterator to the last element
<code>.crend()</code>	Constant reverse iterator to the position one before the first element

```

1 // How to find size of standard C-array
2 int32_t c_arr[] = {0, 1, 5, 3};
3 size_t c_arr_size = sizeof(c_arr) / sizeof(c_arr[0]); // stores 4
4 // How to find size of C++ std::array<>
5 std::array<int32_t, 4> cpp_arr = {0, 1, 5, 3};
6 size_t cpp_arr_size = cpp_arr.size(); // stores 4

```

- C-array size = total size of all elements divided by the size of one element. `std::array<>` does this for you with `.size()`.
- Unlike a C-style array, a `std::array<>` can be treated like a fundamental type: direct assignment/copy with `operator=`, and **value semantics** (can be passed into or returned from a function by value).
- If you need a fixed-size array in C++, prefer `std::array<>` over a C-style array for its cleaner interface and extra features. However, a `std::vector<>` (chapter 7) is usually the better choice anyway.

REMARK: LOOP BOUNDS (IMPORTANT, AND ALWAYS YOUR RESPONSIBILITY)

Bounds checking is not automatic. C++ arrays are **0-indexed**, so the last element has index `SIZE - 1`.

1. Going off the end. Replace `<=` with `<`:

```

// INCORRECT
for (size_t i = 0; i <= SIZE; ++i) {
    arr[i] = i;
}

```

```

// CORRECT
for (size_t i = 0; i < SIZE; ++i) {
    arr[i] = i;
}

```

2. Using index `i + 1` in the body. Stop before `SIZE - 1`, since `i + 1 = SIZE` when `i` reaches `SIZE - 1`, and `SIZE` is not a valid index:

```

// INCORRECT
for (size_t i = 0; i < SIZE; ++i) {
    arr[i] = arr[i + 1];
}

```

```

// CORRECT
for (size_t i = 0; i < SIZE - 1; ++i) {
    arr[i] = arr[i + 1];
}

```

3. Using index `i - 1` in the body. You need an extra iteration to visit every element, so use `<=` and start at 1 (since `i - 1` is invalid when `i` is 0):

```

// INCORRECT
for (size_t i = 1; i < SIZE; ++i) {
    arr[i - 1] = i;
}

```

```

// CORRECT
for (size_t i = 1; i <= SIZE; ++i) {
    arr[i - 1] = i;
}

```

6.3 Storing Multidimensional Data in Arrays

1-D layout

```
int32_t arr_1D[12] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
```

0	1	2	3	4	5	6	7	8	9	10	11
0x100	0x104	0x108	0x10c	0x110	0x114	0x118	0x11c	0x120	0x124	0x128	0x12c

Elements are contiguous. Addresses are arbitrary and separated by 4, the size of a 32-bit integer.

2-D arrays

```
int32_t arr_2D[num_rows][num_cols];
```

```
int32_t arr_2D[3][4] = { {0, 1, 2, 3}, {4, 5, 6, 7}, {8, 9, 10, 11} };
```

- Row 0: 0, 1, 2, 3 | Row 1: 4, 5, 6, 7 | Row 2: 8, 9, 10, 11.
- Access with `[]` twice: **first** bracket is the **row** index, **second** is the **column** index. `arr_2D[1][2]` retrieves 6.
- Because memory chips are 1-dimensional, a 2-D array is laid out in memory as if it were a 1-D array, in **row-major order** — exactly the same layout shown above.

Index conversion formulas

FORMULAS

2-D to 1-D: $1\text{-D_index} = 2\text{-D_row_index} \times \text{num_columns} + 2\text{-D_column_index}$

1-D to 2-D: $2\text{-D_row_index} = 1\text{-D_index} / \text{num_columns}$ $2\text{-D_column_index} = 1\text{-D_index} \% \text{num_columns}$

`%` is the modulo operation, the remainder of a division ($11 \% 5 = 1$). Integer division is used for the row index, so the decimal is truncated.

- When you access an element of a 2-D array, the compiler converts the row and column indices into the corresponding 1-D index. For `arr_2D[1][2]`: $1 \times 4 + 2 = 6$, so index 6 of the underlying 1-D array is accessed.

EXAMPLE 6.1 — 2-D TO 1-D

A 2-D array has 14 rows and 11 columns. The element at row 7, column 9 is accessed. What is its index in the underlying 1-D array?

$1\text{-D_index} = \text{row} \times \text{num_columns} + \text{column} = 7 \times 11 + 9 = 77 + 9 = 86$

EXAMPLE 6.2 — 1-D TO 2-D

A 1-D array of size 126 stores 2-D data with 7 rows and 18 columns. What row and column is index 59?

$\text{row} = 59 / 18 = 3$ (integer division truncates) $\text{column} = 59 \% 18 = 5$

Index 59 corresponds to row 3, column 5.

- **Summary:** regardless of dimension, all arrays look like 1-D arrays in memory. Any n -dimensional array can be converted to a 1-D array using arithmetic operations.

6.4 Heap-Allocated Arrays

The limitation of the stack

- Stack-allocated arrays are **automatically deallocated** once they go out of scope.
- **Limitation:** to declare an array on the stack, its size must be a **constant known at compile time**. This breaks when you do not know the size until runtime:

```

1 size_t size;
2 std::cin >> size;
3 int32_t arr[size];    // size is a runtime value: not allowed by the C++ standard

```

- Some compilers support this, but you should **not** declare a stack array without a compile-time size: if the user enters an absurdly large size, the program exceeds available stack space and crashes immediately.

FOOTNOTE

Variable length stack arrays were permitted in C99 but discontinued in later versions. To use a variable for a stack array size (e.g. `int arr[size]`), you must explicitly initialize the variable and declare it `constexpr`, which specifies that its value can be evaluated at compile time.

Allocating on the heap: new and delete

- The **heap** is a portion of memory that is **not managed automatically**; you are responsible for deallocating anything you allocate there.
- `new` allocates; `delete` deallocates. For arrays, deallocate with `delete[]` (square brackets).
- Failing to deallocate causes a **memory leak**: memory that is no longer accessible still remains in the heap.

```

1 size_t size;
2 std::cin >> size;
3 int32_t* arr = new int32_t[size];
4 /* ... do stuff with array ... */
5 delete[] arr;

```

2-D heap-allocated arrays

- Allocate the **outer dimension (rows)** first, then the **inner dimension (columns)**.

```

1 size_t rows, cols;
2 std::cout << "Enter the number of rows: " << std::endl;
3 std::cin >> rows;
4 std::cout << "Enter the number of columns: " << std::endl;
5 std::cin >> cols;
6 int32_t** arr = new int32_t*[rows];    // array of pointers to other arrays
7 for (size_t r = 0; r < rows; ++r) {
8     arr[r] = new int32_t[cols];        // each row initialized individually
9 } // for r
10 int32_t val = 0;
11 for (size_t r = 0; r < rows; ++r) {    // rows in the OUTER loop = faster
12     for (size_t c = 0; c < cols; ++c) {
13         arr[r][c] = val++;
14     } // for c
15 } // for r
16 /* ... do stuff with the array ... */
17 for (size_t r = 0; r < rows; ++r) {    // delete in the OPPOSITE order of new
18     delete[] arr[r];
19 } // for r
20 delete[] arr;

```

`int** arr = new int*[rows];` `arr` is a pointer to an array on the heap; that array stores pointers to `int` arrays

```

arr[0] → [ ] [ ] [ ] [ ] [ ] [ ] [ ]
arr[1] → [ ] [ ] [ ] [ ] [ ] [ ] [ ]
arr[2] → [ ] [ ] [ ] [ ] [ ] [ ] [ ]

```

The `for` loop initializes each row array with `cols` elements: `arr[r] = new int[cols];`

Caching and loop order

- When looping through a multidimensional array, it is **faster to loop through the outermost dimension (rows) in the outermost loop**.

- This is due to **caching**: it is much more efficient to sequentially access items that are **closer together in memory** than items that are farther apart.

Loop rows, then columns	Loop columns, then rows
Faster — memory accesses are contiguous	Slower — memory accesses are not contiguous

Deleting in reverse order

- `delete` operations must be called in the **opposite order** from the `new` calls.
- Since the array of row pointers (`arr`) was created before the individual row arrays, you must iterate through `arr` and delete all row arrays **before** deleting `arr` itself.

Stack vs heap arrays: summary

	Stack-allocated	Heap-allocated (dynamic, pointer-based)
Deallocation	Automatic when it goes out of scope	Programmer must deallocate before the program ends
Size	Must be known at compile time; limited in size	Can be determined at runtime; flexible
Element access	One memory access regardless of dimension (the compiler does the math to find the object's location)	More than one memory access for larger dimensions: find the pointer to the correct row array, then the element within that row
Verdict	Not very versatile	More flexible; use when array size cannot be determined until runtime

- Heap arrays can be messy to work with. `std::vector<>` (chapter 7) abstracts these details away: it gives all the functionality of a dynamically allocated array, handles allocation and deallocation for you, and can **grow** based on how much data it must store.

REMARK: DYNAMIC MEMORY DISCIPLINE

- Whoever calls `new` is also responsible for calling `delete`.
- Add the `delete` **whenever you call `new`, before you do any other work**. This ensures you do not forget and leak memory that is hard to track down.
- After deleting a pointer to dynamic memory, it is good practice to set it to `nullptr`:

```
1 delete ptr;
2 ptr = nullptr;
```

- This safeguards against unintentional use of a deleted value and protects against **double deletes**: `delete nullptr` is safe and does nothing, but calling `delete ptr` twice could crash your program.

NOTE: SMART POINTERS

To avoid the hassle of `new` and `delete`, use smart pointers from the `<memory>` library (`std::unique_ptr<>`, `std::shared_ptr<>`, `std::weak_ptr<>`). They are wrappers over normal pointers that provide enhanced functionality and automatic memory management. Covered in chapter 27.

6.5 Copying with Pointers

- Goal: copy data from a **source** array into a **destination** array. Assume the destination is large enough to fit all the data.
- Given three pointers:

```
1 int32_t* first;    // pointer to the first element of the source array
2 int32_t* last;     // pointer ONE PAST the last element of the source array
3 int32_t* dest;     // pointer to the first position of the destination array
```

1. Dereference `first` and assign it to `dest` (`*dest = *first`).

2. Increment `first` to move to the next element of the source array (`first++`).
3. Increment `dest` to move to the next position of the destination array (`dest++`).
4. Repeat for all elements, stopping when `first` reaches `last`.

```
while (first != last) {
    *dest++ = *first++;
}
```

```
for (; first != last; ++first, ++dest) {
    *dest = *first;
}
```

- In the while loop, the **postfix** operator (`++` after the name) means incrementing happens **after** the dereference and assignment.
- The loop runs until `first == last`, each time copying the value pointed to by `first` into the position pointed to by `dest`. It terminates once `first` reaches `last`, which points one past the end of the source.
- Use `first != last`, **not** `first < last`: only `!=` is guaranteed to work on all types of containers.
- This method extends across different containers using **iterators** (chapter 11). Pointers and iterators are preferable to indexing because they are more generic: the same code works for multiple container types, including ones that do not support indexing.

NOT TESTED — you do not need to understand how `memcpy()` works

You will see it a lot in upper-level C++ classes, so it does not hurt to be familiar with it.

```
void* memcpy(void* dest, const void* source, size_t num);
```

- Found in the `<cstring>` library. Copies `num` **bytes** from the object pointed to by `source` to the object pointed to by `dest`.
- The underlying types are irrelevant; data is copied as **raw bytes**.
- **Undefined behavior** if `source` and `dest` overlap, or if either points to an invalid address or `nullptr`.
- A `void*` is a generic pointer with no associated type; it can reference an object of any data type.

```
1 int32_t src[] = {1, 2, 3, 4, 5};
2 int32_t dst[5];
3 std::memcpy(dst, src, sizeof(src));
4 for (size_t i = 0; i < 5; ++i) {
5     std::cout << dst[i] << " "; // prints 1 2 3 4 5
6 } // for i
```

- `sizeof()` returns the number of bytes a data type or object occupies in memory. `sizeof(src)` gives the total size of `src` so `memcpy()` copies the correct number of bytes.

6.6 Sequential and Random Access

- Two access methods for iteration that a container may support:
- **Random access**: you can access **any arbitrary element as efficiently as any other**, at any point in time, regardless of the size of the sequence.
- **Sequential access**: elements are visited in a **predetermined order**, where each element is accessed directly after its predecessor.

Random access – user wants the 5th element; can access it directly in constant time

				5th			
--	--	--	--	-----	--	--	--

Sequential access – user wants the 5th element; must iterate through all elements before it

1	2	3	4	5th			
---	---	---	---	-----	--	--	--

- An **array supports random access**: any item can be accessed directly in $\Theta(1)$ time using `operator[]`.
 - This works because array elements are **homogeneous in size** and **contiguous in memory**, so the position of any element can be computed with simple pointer arithmetic.

- A container does **not** support efficient random access if its elements must be accessed in a predetermined sequence. For such containers, the entire container must be read from the beginning to the desired element; there is **no way to calculate the memory location of a random element directly**, so there is no guarantee an element is as easy to access as any other.
- Example of a sequential container: a **linked list** (chapter 8). Without supporting data structures, accessing the arbitrary n th element requires iterating through the first n elements first.

6.7 Data Storage and Retrieval

What to store in a container: values or pointers

Storing by value: the container holds the data itself

0	1	2	3	4	5
---	---	---	---	---	---

Storing by pointer: the container holds addresses that point elsewhere

0x5620	0x573c	0x5818	0x59f4	0x5ad0	0x5bec
--------	--------	--------	--------	--------	--------

Storing by value

- The container has **full ownership** of the memory used to store its data and is responsible for managing it.
- To insert or delete elements, ask the container to do it via an internal operation (such as an `insert()` member function).
- **Most common** approach, but can be **expensive to copy** if the stored objects are large.

Storing by pointer

- The container **no longer has direct ownership** of the data and is not responsible for allocating or deallocating the data values — only the pointers.
- **Dangerous:** the container provides no direct protection, so the data can be modified elsewhere without the container's permission. A container of pointers can easily be **invalidated** if something else deletes what its pointers point to.
- Useful for containers of **C-strings (`char*`)** and for **shared data** (e.g. one master container that stores the data plus smaller containers that access it in different ways through pointers, ensuring the data only exists in memory once).

What to retrieve from a container

Suppose the user requests an element (e.g. via `operator[]`). Three options:

1. **Return a copy.** Inefficient: if the objects are large, every access is slow because a copy is made each time.
2. **Return a pointer.** No copies are needed since an address is returned, but it is **unsafe**: with a pointer to the container's contents, the user could modify anything they want, iterate off the end and access memory they should not, or even delete the data entirely. Returning pointers to private data generally is not ideal.
3. **Return a reference. Typically the best.**
 - Faster than returning by value, since no copy is made.
 - Safer than pointers, because references **cannot be modified** — a user cannot use a reference to maliciously access other memory without permission.
 - Return a **`const` reference** if you want to prevent the user from modifying the contents.

What to store	Value	Pointer
Example	<code>T data;</code>	<code>T* data;</code>
Data ownership	Container	Container or other object
Drawbacks	Large objects can take time to copy	Unsafe; can be modified elsewhere or invalidated
Usage	Most common	Used for <code>char*</code> , shared data

What to retrieve	Value	Pointer, Const Pointer	Reference, Const Reference
E.g.	<code>T get_elt(size_t idx);</code>	<code>T* get_elt(size_t idx);</code>	<code>T& get_elt(size_t idx);</code>
Notes	Costly for copying large objects	Unsafe, can be used to access prohibited memory	Usually a good choice; safer than pointers and faster than copies

BEST PRACTICE FOR A TEMPLATED GETTER

If you do **not** want the container's data to be modified, the ideal prototype is `const TYPE& get_element(int);` — the `const` prevents modification, and the reference avoids a copy.

6.8 Implementing an Array-Based Container

6.8.1 A blueprint of the custom Array container

- Goal: manage a dynamically allocated array while abstracting away memory allocation from the user.
- Motivation:
 - C-style arrays have **fixed size**: adding a 5th element to a 4-element array requires reallocating a new, larger array.
 - Copying/assigning C-style arrays is cumbersome, requiring **deep copies** of all elements.
- Since the container is a custom object, define it with a C++ **class**.

```

1  template <typename T>
2  class Array {
3      T* data;          // Array data (points to a heap-allocated array)
4      size_t length;    // Array size (number of elements)
5  public:
6      Array(size_t len = 0) : length{len} {
7          data = (len ? new T[len] : nullptr);
8      } // Array()
9
10     size_t size() const {
11         return length;
12     } // size()
13 };

```

- **Line 3**: `data` points to a heap-allocated array holding the container's data. **Line 4**: `length` stores the number of elements. Both are **private**.
- **Line 6**: the constructor lets the user pick any size by passing in `len`.
- **Line 7**: the ternary ensures that if `len` is non-zero, `data = new T[len]`; otherwise `data = nullptr`.
- **Line 10**: `size()` returns the length. It is declared `const` because it does not modify any member variables.

```

Array<int32_t> a{5};    // length = 5, data = new T[5]
Array<int32_t> b;       // len not specified, so default argument len = 0 is used;
                        // length = 0 and data = nullptr
std::cout << a.size() << std::endl; // prints 5

```

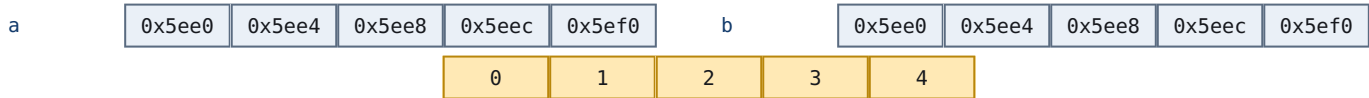
The problem: shallow copies

```

Array<int32_t> a{5};    // initialize 'a' with length 5
Array<int32_t> b = a;   // initialize 'b' with contents of 'a'

```

- To initialize `b`, all member variables of `a` are copied one by one. But since `data` is a **pointer**, the **pointer is copied and not the data objects themselves**. Both `a` and `b` end up sharing the same data.



Both containers point at the same memory.

CONSEQUENCES

- Modifying the contents of `a` also changes the contents of `b`.
- Deallocating `a`'s memory also deallocates `b`'s.
- Accessing an element of `b` after `a` is deallocated accesses **invalid memory**.
- Deallocating `b` after `a` is deallocated is a **double delete** that could crash the program.
- Fix: implement a custom assignment operator that copies the **data objects** themselves, not just the pointers.

6.8.2 The Big Three

RULE

If you are designing a class that **manages dynamic memory** as its data, you should also define the **Big Three** (also called the "Rule of Three") as member functions:

1. **Destructor** — `~Array()`
2. **Copy constructor** — `Array(const Array& other)`
3. **Overloaded assignment operator** — `Array& operator=(const Array& rhs)`

NOT TESTED — the "Big Five" additions (see also section 6.9)

- You may see this rule stated as the **Big Five**, adding:
 1. **Move constructor** — `Array(Array&& other)`
 2. **Overloaded move assignment operator** — `Array& operator=(Array&& rhs)`
- These **transfer ownership** of data from one object to another, rather than copying the data, assigning the copy, and discarding the original.
- The double ampersand (`&&`) indicates an **rvalue reference**: a temporary, unnamed object created during execution.
- Mnemonic: the "r" in rvalue = "right-hand side". Rvalues can never go on the left-hand side of an assignment, because they are temporary and often cannot last beyond the statement they are created in.

```
int32_t a = 10;      // '10' is an rvalue
int32_t b = 20;      // '20' is an rvalue
int32_t c = a + b;    // 'a + b' is an rvalue
int32_t d = a;        // 'a' NOT an rvalue, is a named user-defined variable
```

- They accept an rvalue argument (`&&other`) and can only be invoked if an rvalue is present. Transferring an rvalue's ownership is harmless since it would otherwise be destroyed.
- If an object is assigned or copy constructed from an rvalue and these are defined, the object **claims ownership** of the rvalue's data during construction instead of copying. This speeds things up for large objects.
- If **no rvalue is present**, they are **not** automatically invoked, because non-rvalues are not temporary and may be used later, so transferring their data is unsafe. To move such a variable you must explicitly call `std::move()`:

```
Array<int32_t> a{5};
Array<int32_t> b = std::move(a); // a transfers ownership of its data to b
```

- After the move, `a` **ends up in an unspecified state** and should not be used until it is assigned a new value.

```

1  template <typename T>
2  class Array {
3      T* data;           // Array data
4      size_t length;     // Array size
5  public:
6      // Constructor
7      Array(size_t len = 0) : length{len} {
8          data = (len ? new T[len] : nullptr);
9      } // Array()
10     // Destructor
11     ~Array();
12     // Copy Constructor
13     Array(const Array& other);
14     // Overloaded Assignment Operator
15     Array& operator=(const Array& rhs);
16     // Move Constructor
17     Array(Array&& other) = delete;
18     // Overloaded Move Assignment Operator
19     Array& operator=(Array&& rhs) = delete;
20     size_t size() const {
21         return length;
22     } // size()
23 };

```

- The move members are disabled with `= delete` on lines 21 and 24. In general, setting any member function to `delete` **suppresses** the function and disallows it from being invoked.

6.8.3 Implementing the destructor

- Deallocates all dynamic memory owned by the container by calling `delete[]` on the heap-allocated `data` array.

```

1  ~Array() {
2      delete[] data;    // delete dynamic memory owned by container
3      data = nullptr;   // safeguard against usage/double delete
4  } // ~Array()

```

- **Time complexity: $\Theta(1)$** , since both lines take 1 step (assuming type `T` does not have a destructor of its own).

6.8.4 Implementing the copy constructor

- A copy constructor:
 - has **no return value**;
 - has the **same name as the class**;
 - accepts a **reference** to the object it copies from.
- **The reference is essential:** without it, a copy of the object would have to be made **before the copy constructor can even run**, which is not allowed (it would call the copy constructor you are trying to implement, causing an error).
- The body must explicitly copy each data element one by one, not just the `data` pointer.

```

1  Array(const Array& other) {
2      data = new T[other.length];
3      length = other.length;
4      // make copies of all the elements in the data array
5      for (size_t i = 0; i < length; ++i) {
6          data[i] = other.data[i];
7      } // for i
8  } // Array()

```

Simplified by initializing in a **member-initializer list** instead of the function body:

```

1 Array(const Array& other)
2   : data{new T[other.length]}, length{other.length} {
3   for (size_t i = 0; i < length; ++i) {
4     data[i] = other.data[i];
5   } // for i
6 } // Array()

```

- **Time complexity:** $\Theta(n)$, where n is the number of elements, since every element must be visited.

6.8.5 Implementing the overloaded assignment operator

Attempt 1: copy all data elements from the source Array to the destination Array.

```

1 Array& operator=(const Array& rhs) {
3   delete[] data;           // delete current data
5   length = rhs.length;    // set length and data pointer
6   data = new T[length];
8   for (size_t i = 0; i < length; ++i) { // copy contents of rhs over
9     data[i] = rhs.data[i];
10  } // for i
11  return *this;
12 } // operator=()

```

- **Problem:** fails if someone self-assigns (`a = a`) — the data is deleted before it can be copied.

Attempt 2: add a self-assignment check by comparing memory addresses.

```

1 Array& operator=(const Array& rhs) {
3   if (this == &rhs) {      // don't do anything if self-assigning
4     return *this;
5   } // if
7   delete[] data;           // delete current data
9   length = rhs.length;    // set length and data pointer
10  data = new T[length];
12  for (size_t i = 0; i < length; ++i) {
13    data[i] = rhs.data[i];
14  } // for i
15  return *this;
16 } // operator=()

```

TWO REMAINING PROBLEMS

1. The `this == &rhs` check runs **every single time**, even though self-assignment rarely happens if the programmer knows what they are doing.
2. If memory allocation on line 10 fails, an exception is thrown and the operation terminates. Since the array was already deallocated on line 7, **all the original data is lost**, `data` points to deleted memory, and `length` is wrong. The Array is **corrupted**. If the assignment fails, we still want to retain the original state.

The copy-swap method (the good way)

- Creates a **temporary copy** of the new Array and **swaps** the contents of the old Array with it. The old contents are automatically deallocated when the temporary goes out of scope.
- Fixes both problems:
 - **No self-assignment check is necessary**, which makes the code cleaner.
 - The original data is retained until the assignment succeeds. If assignment fails and an exception is thrown, the failed operation has **no side effects** on the existing data. This safety property is called a **strong exception guarantee**.
- Copy-swap leverages the existing **copy constructor** and **destructor**.

```

1 void swap_array(Array& other) {
2     std::swap(data, other.data);
3     std::swap(length, other.length);
4 } // swap_array()

1 Array& operator=(const Array& rhs) {
2     // create temporary Array storing the contents of the new Array
3     // a deep copy is made since this uses the copy constructor
4     Array temp{rhs};
5     // swap the original data with temp's data
6     swap_array(temp);
7     // old data is now in temp; updated data is now in the current Array (this)
8     return *this;
9 } // operator=()
10 // temp goes out of scope, so the old data is cleaned up by the destructor

```

Visualization of `arr1 = arr2`

Start:

arr1 (4) →

1	2	3	4
---	---	---	---

 arr2 (5) →

5	6	7	8	9
---	---	---	---	---

Line 4 – deep copy of arr2 into local variable `temp`:

temp (5) →

5	6	7	8	9
---	---	---	---	---

Line 6 – contents of arr1 and temp are swapped:

arr1 (5) →

5	6	7	8	9
---	---	---	---	---

 temp (4) →

1	2	3	4
---	---	---	---

After line 9 the function goes out of scope; `temp` (holding arr1's old data) is automatically cleaned up by the Array destructor.

- Self-assignment (`arr1 = arr1`) just swaps the Array with a copy of its own data, so no check is needed.
- If allocation had failed, the exception would be thrown on **line 4**, **before** the original Array is modified on line 6, so the data is never corrupted.
- Line 8 returns `*this`, a reference to the Array that was just assigned to.
- **Time complexity:** $\Theta(n)$, where n is the size of `rhs`, since a deep copy is done.

6.8.6 Implementing the subscript operator

- `operator[]` should **return a reference**, since references are faster than copies and safer than pointers.
- It checks whether the index is valid, then indexes the underlying data array. On an invalid index it throws a `std::runtime_error` (we do not want to return a reference to something the user can unintentionally modify).

```

1 T& operator[](size_t idx) {
2     if (idx < length) {
3         return data[idx];
4     } // if
5     throw std::runtime_error{"Invalid index provided to operator[]"};
6 } // operator[]()

```

This handles indexing correctly. The following prints `0 1 2 3 4`:

```

1 Array<int> a{5}; // init Array of size 5
2 for (size_t i = 0; i < a.size(); ++i) {
3     a[i] = i;
4     std::cout << a[i] << " ";
5 } // for i

```

But this fails:

```

6  const Array<int> b = a;
7  for (size_t i = 0; i < b.size(); ++i) {
8      std::cout << b[i] << " ";
9  } // for i

```

error: passing 'const Array' as 'this' argument discards qualifiers [-fpermissive]

- **Why:** a `const` object cannot call a non-const member function (chapter 1). Since `b` is a `const Array`, it cannot call the non-const `operator[]`.
- Fix: also implement a **`const` version**:

```

1  const T& operator[](size_t idx) const {
2      if (idx < length) {
3          return data[idx];
4      } // if
5      throw std::runtime_error{"Invalid index provided to operator[]"};
6  } // operator[]()

```

- **Both versions must be defined** for indexing to work on const and non-const Array objects.
- The **non-const** version is invoked for a non-const Array; the **const** version is invoked for a const Array.

KEY POINT

Which version runs depends **only on whether the object is const**, not on whether the call actually modifies anything. Even if a call to `operator[]` does not modify a value, the **non-const** version still runs if the container itself is not const.

REMARK: DEFINING MEMBER FUNCTIONS OUTSIDE THE CLASS

- The functions above were written as if implemented **inside** the class definition.
- To define a member function outside the class, use the **scope resolution operator** (`::`) to identify the class it belongs to, with template declarations as needed. It goes **before the function name but after the return type**.

```

1  template <typename T>
2  class Array {
3      T* data;
4      size_t length;
5  public:
6      Array(size_t len = 0);
7      ~Array();
8      Array(const Array& other);
9      void swap_array(Array& other);
10     Array& operator=(const Array& rhs);
11     T& operator[](size_t idx);
12     const T& operator[](size_t idx) const;
13     size_t size() const;
14 };
15
16 template <typename T>
17 Array<T>::Array(size_t len) : length{len} {
18     data = (len ? new T[len] : nullptr);
19 } // Array()
20
21 template <typename T>
22 Array<T>::~~Array() {
23     delete[] data;
24     data = nullptr;
25 } // ~Array()
26
27 template <typename T>
28 Array<T>::Array(const Array& other)
29     : data{new T[other.length]}, length{other.length} {
30     for (size_t i = 0; i < length; ++i) {
31         data[i] = other.data[i];
32     } // for i
33 } // Array()
34
35 template <typename T>
36 void Array<T>::swap_array(Array& other) {
37     std::swap(data, other.data);
38     std::swap(length, other.length);
39 } // swap_array()
40
41 template <typename T>
42 Array<T>& Array<T>::operator=(const Array& rhs) {
43     Array temp{rhs};
44     swap_array(temp);
45     return *this;
46 } // operator=()
47
48 template <typename T>
49 T& Array<T>::operator[](size_t idx) {
50     if (idx < length) { return data[idx]; }
51     throw std::runtime_error{"Invalid index provided to operator[]"};
52 } // operator[]()
53
54 template <typename T>
55 const T& Array<T>::operator[](size_t idx) const {
56     if (idx < length) { return data[idx]; }
57     throw std::runtime_error{"Invalid index provided to operator[]"};
58 } // operator[]()
59
60 template <typename T>
61 size_t Array<T>::size() const {
62     return length;
63 } // size()

```

6.8.7 Implementing insert

INSERTION IS COMPLICATED FOR TWO REASONS

1. Elements in an array-based container must be stored **contiguously** in memory.
2. Once you allocate a data array, its **size cannot change**; for more space you must allocate a new array and move the data over.

Problem 1: shifting

To insert 2 between 1 and 3, the 2 must end up directly after 1 and directly before 3, so **every element after the insertion point must be shifted one to the right**.

before	1	3	4	5	→ shift right →
after	1	2	3	4	5

```

1  template <typename T>
2  bool insert(size_t idx, T val) {
3      if (idx < length) {
4          for (size_t i = length - 1; i > idx; --i) {
5              data[i] = data[i - 1];
6          } // for i
7          data[idx] = val;
8          return true;
9      } // if
10     return false;
11 } // insert()

```

- Takes the **index of insertion** and the **value**; shifts everything after the insertion point and inserts at the specified index (`insert(1, 2)` inserts 2 at index 1). Returns `true` on success, `false` otherwise.

Case	Complexity	Reason (idx valid, array has n elements)
Best case	$\Theta(1)$	Inserting at the very end of the array; no shifting is needed
Worst case	$\Theta(n)$	Inserting at the very beginning ; all elements after it must be shifted
Average case	$\Theta(n)$	Averaged over all cases, $n/2$ elements shift, and $\Theta(n/2) = \Theta(n)$ after dropping coefficients

Problem 2: fixed capacity and reallocation

- Once an array is allocated, its size is fixed. If `data = new int32_t[4]`, that array cannot expand, so the insert above would not even work when the array is full.
- To add a 5th element to a 4-element array, one approach is:
 1. Allocate an entirely new array that can hold 5 elements.
 2. Copy over the original 4 elements.
 3. Deallocate the old array and reset the pointer.
 4. Add the 5th element.

WHY GROW-BY-ONE IS NOT PERFORMANT

Copying takes $\Theta(n)$ time for n elements, and you would copy **every time** an element is inserted beyond the array's capacity. With 1,000 elements, adding the 1,001st and 1,002nd requires **two array allocations and 2,001 copies** (1,000 on the first allocation, 1,001 on the second).

The fix: double the capacity

- Plan ahead and allocate **more memory than needed**. Instead of incrementing capacity by 1, **double the capacity** on reallocation (4 → 8 instead of 4 → 5).
- Then the 6th, 7th, and 8th elements require no copying. Adding the 9th triggers a reallocation with elements copied over, but the new capacity doubles to 16, allowing the 10th through 16th elements to be inserted in constant time with no additional copying overhead.

- Because more space is allocated than needed, **size and capacity may differ**, so both must be tracked:
 - **capacity** tells you when you must reallocate (the maximum number of elements the underlying array can hold).
 - **size** tells you the index of the last valid element (the number of valid elements). If capacity is 8 but only 5 elements are valid, the user should not be accessing indices 5 to 7.

```

1  template <typename T>
2  class Array {
3      T* data;           // pointer to Array data
4      size_t size;       // number of valid elements in 'data' array
5      size_t capacity;   // capacity of underlying 'data' array
6      ...
7  };

```

- **Why not triple or quadruple?** You could, but it would be **wasteful of memory**: there is no guarantee all that memory will be used, and the performance improvement is **asymptotically insignificant**.

Stopping here

- More could be added (deleting elements, manually changing capacity), but we stop for two reasons:
 - Several of these features get quite complicated (insertion and deletion both require shifting elements).
 - More importantly, it is not necessary: **C++ already provides `std::vector`**, which manages dynamic memory and does everything implemented here and much more. No need to reinvent the wheel. Covered in chapter 7.

6.9 Move Semantics

NOT TESTED — this entire section is not taught in this class

This is core C++ knowledge and worth reading after the class. Move semantics are **not banned on the autograder**, so you may use them in assignments if you understand them, but you will not need them to do well.

Move semantics allow resources to be **transferred** between objects instead of copied, avoiding unnecessary copies of temporary objects that will not exist much longer.

6.9.1 Lvalues and rvalues

- **Lvalue** ("left-hand" value): a **named** object with an **identifiable location in memory**. Also known as a **locator value**. Can go on the left-hand side of an assignment.
- **Rvalue** ("right-hand" value): a **temporary, unnamed** object that cannot be assigned to, so it can only go on the right-hand side.

```

int32_t x = 281;    // x is an lvalue (named, has an address); 281 is an rvalue (temporary)

int32_t y = x;      // OK, x and y are both lvalues
281 = x;            // NOT OK, 281 is an rvalue

```

- Lvalues can be assigned from rvalues or other lvalues; rvalues cannot be assigned to at all.
- A function that **returns by value** returns an **rvalue**:

```

1  int32_t get_favorite_class() {
2      static int32_t fav_class = 281;
3      return fav_class;
4  } // get_favorite_class()
5
6  int main() {
7      int32_t x = get_favorite_class(); // x is lvalue, get_favorite_class() is rvalue
8  } // main()

get_favorite_class() = 370; // not OK, it is an rvalue

```

- A function that **returns by reference** returns an **lvalue reference**, which can be assigned to:

```

1 int32_t& get_favorite_class() {
2     static int32_t fav_class = 281;
3     return fav_class;
4 } // get_favorite_class()
6 int32_t main() {
7     get_favorite_class() = 370;    // OK, it is an lvalue reference
8 }

```

- You cannot construct an lvalue reference from an rvalue: `int32_t& x = 281;` is not OK.
- **Exception:** an lvalue reference **can** bind to an rvalue if it is declared `const`: `const int32_t& x = 281;` is OK.

```

void foo(std::string& s) {
    /* ... do stuff ... */
} // foo() -- non-const lvalue ref

```

```

void bar(const std::string& s) {
    /* ... do stuff ... */
} // bar() -- const lvalue ref

```

```

1 std::string str1 = "EECS";
2 std::string str2 = "281";
4 foo(str1);           // OK, str1 is lvalue
5 foo(str2);           // OK, str2 is lvalue
6 foo(str1 + str2);    // NOT OK, str1 + str2 is rvalue
7 foo("EECS 281");     // NOT OK, "EECS 281" is rvalue
9 bar(str1);           // OK
10 bar(str2);           // OK
11 bar(str1 + str2);    // OK, rvalue, but lvalue reference parameter is const
12 bar("EECS 281");     // OK, rvalue, but lvalue reference parameter is const

```

- **Why this matters:** we can treat lvalues and rvalues with different levels of care. An lvalue may still be needed by something else afterward, so we must be careful with its data. An rvalue is temporary, so we can abuse it without consequences — enabling optimizations like **stealing an object's resources** and putting them somewhere else.
- An **rvalue reference** is denoted with a double ampersand (`&&`) and can only take temporary rvalue objects:

```

1 void baz(std::string&& s) { /* ... do stuff ... */ } // only accepts rvalues
4 baz(str1);           // NOT OK, str1 is lvalue
5 baz(str2);           // NOT OK, str2 is lvalue
6 baz(str1 + str2);    // OK, rvalue
7 baz("EECS 281");     // OK, rvalue

```

You can overload a function so rvalues get special optimizations:

```

1 void qux(const std::string& s) {
2     // runs if input is an lvalue
3     // even though rvalues can be accepted as const lvalue references,
4     // rvalues will always be sent to the && version if one exists
6 } // qux()
8 void qux(std::string&& s) {
9     // runs if input is a temporary rvalue
10    // can do rvalue optimizations (e.g., safely steal data from s)
12 } // qux()
14 int main() {
15     std::string str1 = "EECS";
16     std::string str2 = "281";
18     qux(str1);         // first version runs (const& s)
19     qux(str2);         // first version runs (const& s)
20     qux(str1 + str2);  // second version runs (&&s)
21     qux("EECS 281");  // second version runs (&&s)
22 } // main()

```

6.9.2 Implementing the move constructor

```

1  class String {
2      char* data;      // pointer to char array on heap
3      size_t length;   // length of String
4  public:
5      // constructor from char*
6      String(const char* str) {
7          length = strlen(str);
8          data = new char[length + 1];
9          std::cout << "Char Ctor: Heap allocation made" << std::endl;
10         for (size_t i = 0; i < length; ++i) { data[i] = str[i]; }
13         data[length] = '\0';
14     } // String()
15     // copy constructor from another String
16     String(const String& other)
17         : data{new char[other.length + 1]}, length{other.length} {
18         std::cout << "Copy Ctor: Heap allocation made" << std::endl;
19         for (size_t i = 0; i < length; ++i) { data[i] = other.data[i]; }
23         data[length] = '\0';
24         std::cout << "Copy Ctor: String successfully copied" << std::endl;
25     } // String()
27 };

29 class Student {
30     String name;
31 public:
32     Student(const String& name_in) : name{name_in} {}
33 };

38 int main() {
39     Student s{"Alice"};
40 } // main()

```

Output:

```

Char Ctor: Heap allocation made
Copy Ctor: Heap allocation made
Copy Ctor: String successfully copied

```

- **What happened:** one Student holding one String produced **two heap allocations and a copy**, because a temporary rvalue was copied.
 1. On line 39, "Alice" is converted into a temporary `String` object (an rvalue) → first heap allocation (line 8).
 2. That rvalue is passed to the constructor and assigned to `name`, invoking the **copy constructor**, which allocates new memory (line 18) and copies the rvalue's contents into it.
 3. The rvalue is then destructed.
- Instead, we can **transfer** the data from the first String to the Student's `name`, needing only a **single heap allocation**.
- A **move constructor** is a constructor that accepts an rvalue of the same type; it is invoked when an object is initialized to an rvalue. It does not allocate new memory or copy; it safely takes ownership because the rvalue will be destroyed afterward.

```

1 class String {
2     char* data;
3     size_t length;
4 public:
5     // move constructor
6     String(String&& other) noexcept
7         : data{other.data}, length{other.length} { // shallow copy = "steal"
8         other.data = nullptr;
9         other.length = 0;
10        std::cout << "Move Ctor: Data transferred from rvalue" << std::endl;
11    } // String()
12 };

```

- **Setting the rvalue's `data` pointer to `nullptr` is essential.** The rvalue's data gets destructed after the move constructor completes; without this, the data you stole would be deleted along with it.
- **Remark:** move constructors and move assignment operators should be declared `noexcept`, indicating they should not throw any exceptions.

```

1 class Student {
2     String name;
3 public:
4     // constructor (for lvalues)
5     Student(const String& name_in) : name{name_in} {}
6     // constructor (for rvalues)
7     Student(String&& name_in) : name{name_in} {} // still copies!
8 };

```

- **Nothing changes.** Why? Although the constructor on lines 8-9 only runs for rvalue references, the String passed in **gains a name** (`name_in`) and an identifiable memory location (you can take its address). So the rvalue reference **behaves like an lvalue reference inside the function**.
- You must explicitly cast it back into an rvalue with `std::move()`:

```

8 // constructor (for rvalues)
9 Student(String&& name_in) : name{std::move(name_in)} {} // now moves

```

New output:
 Char Ctor: Heap allocation made
 Move Ctor: Data transferred from rvalue

REMARK: WHAT `STD::MOVE()` ACTUALLY DOES

`std::move()` **casts its argument to an rvalue**; it does not physically move anything in memory. It tells the compiler the object can be treated as an rvalue and its data safely stolen. The actual stealing is done in the move constructor or move assignment operator. After calling `std::move()` on an object, it is in an **unspecified state** and should not be used until reassigned.

6.9.3 Implementing the overloaded move assignment operator

- The standard `operator=` must copy the object before assigning it (to do `a = b`, copy `b` and assign it to `a`). If assigning from an rvalue, you can steal instead.

```

7 String& operator=(String&& rhs) noexcept {
8     // don't do anything if self-assigning
9     if (this == &rhs) { return *this; }
10    delete[] data; // delete original data
11    length = rhs.length; // steal length and data pointer from rvalue
12    data = rhs.data;
13    // clear rvalue's data so what you stole isn't deleted on its destruction
14    rhs.length = 0;
15    rhs.data = nullptr;
16 } // operator=()

```

As with the standard assignment operator, the self-assignment check can be removed by using

```
std::swap():
```

```
7   String& operator=(String&& rhs) noexcept {
8       std::swap(this->length, rhs.length);
9       std::swap(this->data, rhs.data);
10      return *this;
11  } // operator=()
```

REMARK: HOW STD::SWAP() IS IMPLEMENTED

Before C++11 (copies):

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
} // swap()
```

C++11 and later (moves, no copies):

```
template <typename T>
void swap(T& a, T& b) {
    T temp = std::move(a);
    a = std::move(b);
    b = std::move(temp);
} // swap()
```

6.9.4 Return value optimization (RVO)

- **Do not use `std::move()` to return a local object from a function**, even though it looks like an optimization. It suppresses **return value optimization**.

Good (RVO can happen):

```
std::string gen_big_string(size_t n) {
    std::string big_string;
    /* ... generate big string ... */
    return big_string;
}
int main() {
    std::string str = gen_big_string(281);
}
```

Bad (RVO is blocked):

```
std::string gen_big_string(size_t n) {
    std::string big_string;
    /* ... generate big string ... */
    return std::move(big_string);
}
int main() {
    std::string str = gen_big_string(281);
}
```

- If a function returns a **local object by value** and the object's type **matches the return type**, the compiler may use RVO to build the object **directly in its intended destination** in memory. No copying or moving is done at all. Because the copy/move is elided, RVO is formally a **copy elision optimization**.
- Calling `std::move()` means you are no longer returning a local object of the same type; you are returning an **rvalue reference** (`std::string&&`), which violates the conditions for RVO. The compiler must then build a separate instance in `gen_big_string()`'s stack frame and transfer its data into `str` in `main()`'s stack frame — strictly worse.

6.9.5 Perfect forwarding and forwarding references

- With **perfect forwarding**, a templated function accepts a **forwarding reference** (also written with `&&`) and uses it with `std::forward<>` to **preserve the original value category** of its argument (lvalue vs rvalue).

```
1  template <typename T>
2  void foo(T&& arg) {
3      bar(std::forward<T>(arg));
4  } // foo
```

- If `arg` is passed into `foo()` as an lvalue reference, it is passed to `bar()` as an lvalue reference; same for rvalue references.
- **Why `std::forward<>` is needed:** all function parameters behave like lvalues even when passed in as rvalues (the argument is a named object), so `arg` is always an lvalue. `std::forward<>` performs a **conditional cast** based on the value category of the original argument:
 - Called on an lvalue reference → does nothing, `bar()` gets the same lvalue reference.
 - Called on an rvalue reference → casts `arg` back into an rvalue before passing it to `bar()`.

- It detects which case applies via **reference collapsing** during template deduction:

Deduced argument type	Resolves to
<code>T& &</code>	<code>T&</code> (lvalue reference)
<code>T& &&</code>	<code>T&</code> (lvalue reference)
<code>T&& &</code>	<code>T&</code> (lvalue reference)
<code>T&& &&</code>	<code>T&&</code> (rvalue reference)

- For `template <typename T> void foo(T&& arg);`
 - Passing an lvalue reference of type `Thing`: `foo(T&&)` deduces to `foo(Thing& &&)`, which resolves to `foo(Thing&)`.
 - Passing an rvalue reference of type `Thing`: `foo(T&&)` deduces to `foo(Thing&& &&)`, which resolves to `foo(Thing&&)`.
- **Value:** avoids excessive copying and lets you write one method supporting many input value types instead of multiple overloads (one for lvalues, one for rvalues, etc.).
- Best use case: when an object must be moved through **multiple function calls** between creation and destination. Many standard library functions use this, e.g. `std::vector<>::emplace_back(Args&&... args)`, which takes constructor arguments and forwards them to the constructor of the object, which is then constructed directly at the back of the vector (chapter 7).

6.10 Valgrind and Memory Debugging Tools

- Whenever you manage memory explicitly there is an inherent risk of memory-related errors: **segmentation faults**, **uninitialized values**, and **memory leaks**.
- **valgrind** detects these issues. Run it as `valgrind ./exec`.

Example 1: off-the-end access, use-after-free, and a leak

```
valgrind.cpp
1 int main() {
2     int32_t* arr = new int32_t[3];
3     for (size_t i = 0; i <= 3; ++i) {
4         arr[i] = i * i;
5         std::cout << arr[i] << " ";
6     } // for i
7     delete[] arr;
8     int32_t val;
9     arr[0] = val;
10    int32_t* leak = new int32_t[100];
11 } // main()
```

- Running it prints `0 1 4 9` and everything looks fine, but valgrind reveals the code is buggy.

Error 1 and 2: invalid write / invalid read

```

==8947== Invalid write of size 4
==8947==    at 0x400814: main (valgrind.cpp:4)
==8947== Address 0x5aaec8c is 0 bytes after a block of size 12 alloc'd
==8947==    at 0x4C2A888: operator new[](unsigned long) (vg_replace_malloc.c:423)
==8947==    by 0x4007E7: main (valgrind.cpp:2)

==8947== Invalid read of size 4
==8947==    at 0x40082A: main (valgrind.cpp:5)
==8947== Address 0x5aaec8c is 0 bytes after a block of size 12 alloc'd
==8947==    by 0x4007E7: main (valgrind.cpp:2)

```

- **Translation:** line 4 tried to **write** 4 bytes to an invalid address, and line 5 tried to **read** 4 bytes from an invalid address. The address is **after a block of size 12**, meaning the program went off the end of that block. The block was allocated on line 2.
- Line 2 allocated a 12-byte block (3 integers × 4 bytes). So `i` is running off the end: `arr[3]` is invalid, but the final iteration accesses it.
- **Fix:** change `<=` to `<` on line 3.

Error 3: writing to freed memory

```

==8947== Invalid write of size 4
==8947==    at 0x400865: main (valgrind.cpp:9)
==8947== Address 0x5aaec80 is 0 bytes inside a block of size 12 free'd
==8947==    at 0x4C2B61D: operator delete[](void*) (vg_replace_malloc.c:621)
==8947==    by 0x40085D: main (valgrind.cpp:7)
==8947== Block was alloc'd at
==8947==    by 0x4007E7: main (valgrind.cpp:2)

```

- **Translation:** line 9 writes 4 bytes to an invalid address. "0 bytes inside a block of size 12 free'd" means the memory was **previously deallocated**.
- Reading the output **bottom-up**: the block was allocated on line 2 and deleted on line 7, so line 9 writes to memory we no longer have access to.

The leak summary

```

==8947== HEAP SUMMARY:
==8947==    in use at exit: 400 bytes in 1 blocks
==8947== total heap usage: 3 allocs, 2 frees, 73,116 bytes allocated

==8947== LEAK SUMMARY:
==8947==    definitely lost: 400 bytes in 1 blocks
==8947==    indirectly lost: 0 bytes in 0 blocks
==8947==    possibly lost: 0 bytes in 0 blocks
==8947==    still reachable: 0 bytes in 0 blocks
==8947==    suppressed: 0 bytes in 0 blocks
==8947== Rerun with --leak-check=full to see details of leaked memory
==8947== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)

```

Category	Meaning
definitely lost	The pointer to a block of dynamic memory is fully lost before the memory is deallocated. Red flag.
indirectly lost	The pointer to that block was located inside another block that was definitely lost. Red flag.
still reachable	Heap memory not explicitly freed at exit, but the program still held a pointer or reference to it. Usually not harmful during development and not a true leak; the OS completes the cleanup.

- A non-zero value for the first two categories should be treated as a red flag: it indicates a possible programming error with memory that was not correctly cleaned up.

- Here, 400 bytes were definitely lost: line 10 allocated 100 integers $\times$ 4 bytes = 400 bytes, and it was never deallocated with `delete[]`, so it was lost when `main()` exited.

Example 2: uninitialized values (a subtler error)

```

1  int32_t* add_100(int32_t n) {
2      int32_t res = 100;
3      res = 100 + n;
4      int32_t *addr = &res;
5      return addr;
6  } // add_100()
8  void print_number(int32_t num) {
9      std::cout << num;
10 } // print_number()
12 int main() {
13     int32_t eecs = 281;
14     int32_t *sum = add_100(eecs);
15     print_number(*sum);
16 } // main()

```

- `add_100()` adds 100 to a number and returns a pointer to it, so `main()` should print $100 + 281 = 381$. But `valgrind` shows memory issues.

```

==9061== Conditional jump or move depends on uninitialised value(s)
==9061==    at 0x4F3698B: std::ostreambuf_iterator<char, ...> std::num_put<...>
==9061==    by 0x4F42B89: std::ostream& std::ostream::_M_insert<long>(long) (ostream.tcc:73)
==9061==    by 0x400704: print_number(int) (valgrind.cpp:9)
==9061==    by 0x400731: main (valgrind.cpp:15)

==9061== Use of uninitialised value of size 8
==9061==    at 0x4F35893: int std::__int_to_char<char, unsigned long>(...)
==9061==    by 0x400704: print_number(int) (valgrind.cpp:9)
==9061==    by 0x400731: main (valgrind.cpp:15)

(two more "Conditional jump or move depends on uninitialised value(s)" blocks)

==9061== HEAP SUMMARY:
==9061==    in use at exit: 0 bytes in 0 blocks
==9061==    total heap usage: 1 allocs, 1 frees, 72,704 bytes allocated

```

- How to read messy output:** look for the lines that **directly reference your own code** (bolded above); they lead you to the source of the error. Most other lines relate to stream operations.
- All errors start at line 15, which calls `print_number()` with `*sum`; `print_number()` calls `std::cout` on line 9. With each call an **uninitialized value** is being passed in, hinting that `sum` is uninitialized.
- The actual bug:** `add_100()` returns `addr`, a pointer to a local variable. `addr` was allocated **on the stack** and went out of scope when `add_100()` exited, so the pointer given to `sum` is out of scope and therefore uninitialized.

REMARK: --TRACK-ORIGINS=YES

If you still cannot find the error, this flag helps locate the source:

```

./valgrind --track-origins=yes ./<executable_name>

==9061==    by 0x400704: print_number(int) (valgrind.cpp:9)
==9061==    by 0x400731: main (valgrind.cpp:15)
==9061== Uninitialised value was created by a stack allocation
==9061==    at 0x400721: main (valgrind.cpp:14)

```

This indicates the uninitialized value was created by a **stack allocation on line 14** (the pointer `sum`).

Chapter 6 Quick Reference

Operation / concept	Result
Access array element by index	$\Theta(1)$ — <code>arr[i]</code> is <code>*(arr + i)</code>
Copy an array of n elements	$\Theta(n)$ — cannot do better; every element must be visited at least once
Insert at the back (capacity available)	$\Theta(1)$ — no other elements are touched
Insert at the front (capacity available)	$\Theta(n)$ — all existing elements shift over by one
Array destructor (<code>delete[]</code>)	$\Theta(1)$
Copy constructor / <code>operator=</code>	$\Theta(n)$ — deep copy
2-D to 1-D index	<code>row * num_columns + column</code>
1-D to 2-D index	<code>row = idx / num_columns, column = idx % num_columns</code>
Big Three	Destructor, copy constructor, overloaded assignment operator — required for any class managing dynamic memory
Copy-swap	Technique for the overloaded assignment operator ; reuses copy constructor and destructor; no self-assignment check; strong exception guarantee
Growing an array	Double the capacity on reallocation; track <code>size</code> and <code>capacity</code> separately
Best getter prototype (read-only)	<code>const TYPE& get_element(int);</code>
Which <code>operator[]</code> runs	Depends only on whether the object is <code>const</code> , not on whether the call modifies anything